

QRay v5 – What’s Actually Working, and How to Build It

A beginner-friendly, implementation-ready guide for the team

QRay Project – Internal Technical Reference

Contents

0. Read This First – What This Document Is	2
1. The Big Picture – Before Any Code	2
1.1 What problem are we solving?	2
1.2 What does “quantum-inspired” actually mean here?	3
1.3 The five-layer architecture	3
1.4 Mathematical formulation (for the record)	3
2. Layer 1: Graph & Encoding	5
2.1 What it does	5
2.2 The one crucial performance trick	5
2.3 Representing a solution	6
3. Layer 2 + 3: The Core Search Engine (This Is the Main Algorithm)	6
3.1 The core idea, explained with an analogy	6
3.2 Step-by-step algorithm	8
3.3 Building the starting solution (greedy nearest-neighbor)	9
3.4 The four destroy operators	10
3.5 Repair: putting removed stops back (use greedy, not regret-based)	11
3.6 Acceptance: the quantum-tunneling formula	12
3.7 Polishing: 2-opt	12
4. Layer 4: Dynamic Time-Slicing (Live Traffic Re-Routing)	13
4.1 The key idea	13
4.2 Why this works, and how much it helps	13
4.3 Implementation checklist	14
4.4 A correction: re-routing must account for where trucks actually are	14
5. Layer 5: Feasibility & Cost Calculation	15
5.1 What “feasible” means	15
5.2 Calculating total cost	15
5.3 The single most important performance lesson in this whole project	15
5.4 Demand overloading: what if one stop’s demand exceeds a truck’s capacity?	16
6. Why Each Piece Actually Works (The Evidence, Briefly)	17
6.1 Why the greedy starting point matters	17
6.2 Why the destroy/repair loop beats plain quantum particle swarm	17

6.3 What's genuinely proven about the "quantum" parts, and what isn't	17
7. Performance Checklist Before You Consider It "Done"	17
7.1 Numba JIT – what we actually measured (don't skip this before enabling it) . .	18
8. What Was Tried and Should NOT Be Re-Implemented	18
9. Appendix A — Full Details of Every Algorithm Tried	19
A.1 Quantum-behaved Particle Swarm Optimization (QPSO) – the original engine .	19
A.2 Chaos/Mutation Escape Mechanism	20
A.3 Quantum-Inspired Genetic Algorithm (QGA)	20
A.4 Quantum-Inspired Ant Colony Optimization (QACO)	20
A.5 Quantum-Annealing-Inspired Local Search (QASA)	20
A.6 Quantum Solution Swarm (QSS)	20
A.7 Classical (non-quantum) controls	21
A.8 Google OR-Tools (external benchmark, not a QRay variant)	21
10. Known Limitations and Potential Flaws	21
11. Quick Reference: Parameters That Were Empirically Tuned	22
11.1 Scale-dependent destroy fraction and gamma factor (re-tuned)	23

0. Read This First – What This Document Is

This document is for teammates who are picking up QRay and need to actually **build** the parts that are proven to work. It assumes you know how to program (any language is fine, examples below use Python-style pseudocode) but assumes **nothing else** – no prior quantum computing knowledge, no prior routing/optimization background.

Everything in this document has been tested empirically across six rounds of benchmarking against Google OR-Tools and classical algorithms. We only include what survived that testing. Where something was tried and failed, it's listed in Section 8 so you don't waste time re-building it.

What you're building, in one sentence: a program that takes a list of delivery stops, a fleet of trucks with limited capacity, and a map of roads with travel times, and produces a set of routes (one per truck) that gets everything delivered as cheaply and quickly as possible – and that can update those routes fast when traffic changes.

1. The Big Picture – Before Any Code

1.1 What problem are we solving?

This is called the **Capacitated Vehicle Routing Problem (CVRP)**. In plain terms:

- You have one **depot** (warehouse) where all trucks start and end.
- You have a list of **stops** (customers/deliveries) that need to be visited exactly once.
- You have a fleet of **trucks**, each with a maximum **capacity** (how much weight/volume it can carry).
- Every stop has a **demand** (how much capacity it uses up).

- You want to split the stops across trucks and decide the order each truck visits its stops, such that:
 1. No truck carries more than its capacity.
 2. Every stop is visited exactly once.
 3. Every truck starts and ends at the depot.
 4. The total distance/time driven across all trucks is as small as possible.

This is a genuinely hard problem – the number of possible ways to split and order stops grows explosively as the number of stops grows (this is why it’s called “NP-hard”). You cannot just try every possibility for anything beyond a handful of stops. That’s why we use **metaheuristics** – smart search strategies that don’t guarantee the perfect answer, but reliably find very good answers, fast.

1.2 What does “quantum-inspired” actually mean here?

Important, say this to anyone who asks: nothing in this project runs on quantum hardware. “Quantum-inspired” means we borrow *mathematical ideas* from quantum physics (probability, uncertainty, tunneling, superposition) and turn them into ordinary formulas that run on a normal computer. This is a legitimate, published area of algorithm research – it is not a marketing trick, but it’s also not “real” quantum computing, and you should never imply otherwise.

1.3 The five-layer architecture

QRay is built as five layers stacked on top of each other. Layers 1 and 5 are “plumbing” – necessary infrastructure that isn’t very interesting on its own. Layers 2, 3, and 4 are where the actual intelligence lives.

Layers 2 and 3 are actually implemented as **one continuous loop** in the working version (they’re separated conceptually, not as separate function calls) – this will make more sense once you see the pseudocode in Section 3.

1.4 Mathematical formulation (for the record)

For documentation completeness, here is the problem stated formally. You don’t need this to code the system (Section 3’s pseudocode is sufficient), but a judge or reviewer may expect to see it written this way.

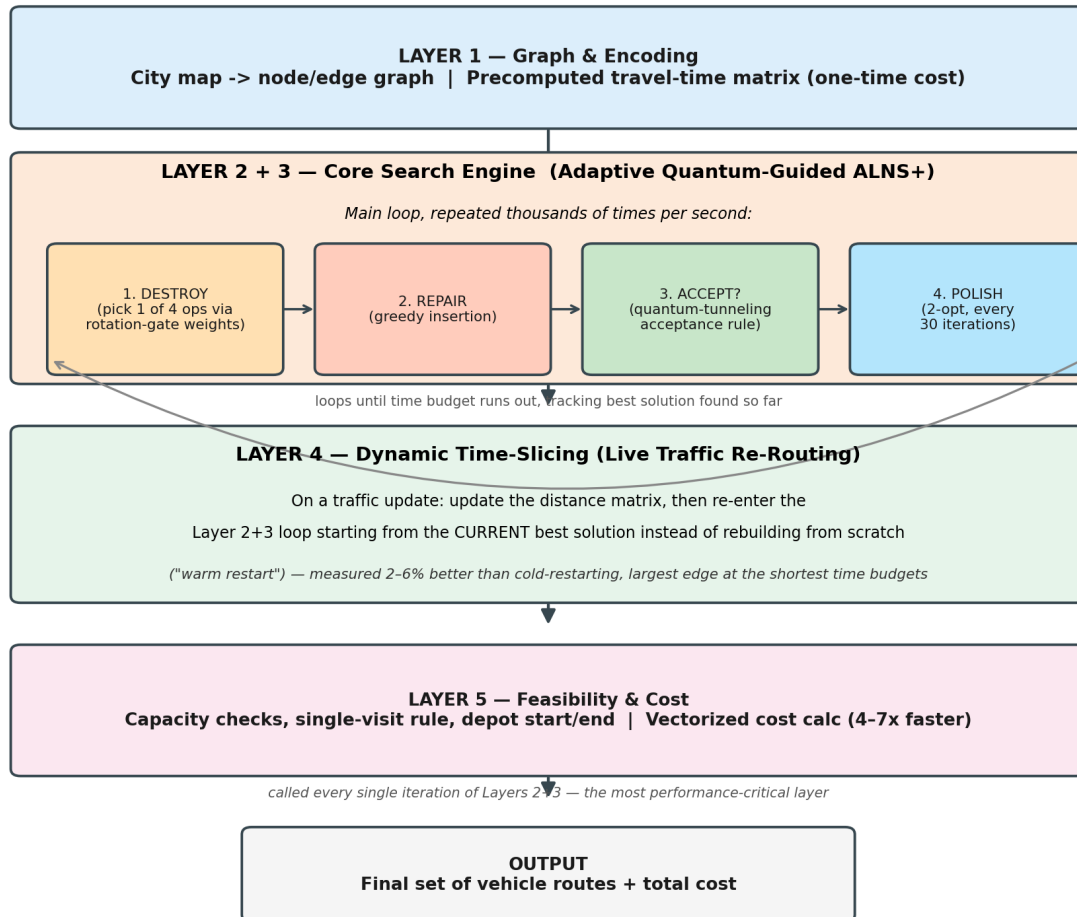
Objective – minimize total distance across all vehicle routes:

$$\min \sum_{k=1}^V \sum_{(i,j) \in R_k} d_{ij}$$

where V is the number of vehicles, R_k is the ordered sequence of edges in vehicle k ’s route, and d_{ij} is the travel time/distance between nodes i and j (from the precomputed matrix in Section 2.2).

Subject to:

QRay v5 — System Architecture



Where the "quantum-inspired" ideas live:

- **Quantum-variance destroy operator:**
shadow-swarm disagreement picks which stops to re-examine
- **Rotation-gate operator weighting:**
compass-needle-style nudge toward operators that keep working
- **Quantum-tunneling acceptance:**
fat-tailed accept-a-worse-move rule (Sec. 3.6 caveat applies)

Everything else (greedy seeding, ALNS destroy/repair structure, warm restart, vectorized feasibility) is classical engineering — and is where most of the proven gains come from.

Figure 1: QRay v5 system architecture

- **Capacity:** $\sum_{i \in R_k} q_i \leq Q_k$ for every vehicle k , where q_i is stop i 's demand and Q_k is vehicle k 's capacity.
- **Single visit:** every stop $i \in \{1, \dots, N\}$ appears in exactly one R_k , exactly once.
- **Depot start/end:** every route begins and ends at node 0 (the depot).

Search-space encoding: a solution is decoded from a vector of real-valued “keys” $\mathbf{x} \in [0, 1]^N$ (one per stop) by sorting stops by key value and splitting the sorted sequence into vehicle routes greedily on capacity overflow. In the current engine (Section 3) the search does not manipulate keys directly – it manipulates route lists via destroy/repair – but the key-based encoding is still used internally for the quantum-variance operator’s “shadow swarm” (Section 3.4.4) and is the same encoding the original QPSO engine (Section 6.2, Appendix A.1) used directly.

Acceptance criterion (Section 3.6), formally:

$$P(\text{accept} \mid \Delta) = \begin{cases} 1 & \Delta < 0 \\ \frac{1}{1 + (\Delta/\Gamma)^2} & \Delta \geq 0 \end{cases} \quad \Gamma(t) = \Gamma_0 \cdot \max(0.02, 1 - t/T)^2$$

where Δ is the cost change of a proposed move, t/T is the fraction of the time budget elapsed, and $\Gamma_0 = 0.08 \times$ (starting solution cost).

2. Layer 1: Graph & Encoding

2.1 What it does

Think of the city as a dot-to-dot puzzle:

- Every stop (and the depot) is a **node** – just a point with an (x, y) coordinate, or a real latitude/longitude if you’re using a real map.
- Every pair of nodes has a **travel time** between them (if you’re using a real road network, this comes from an actual routing engine like OSRM or a maps API; for testing, we just use straight-line distance).

2.2 The one crucial performance trick

Calculate every pairwise travel time once, up front, and store it in a table (a matrix). Do not recalculate “how far is A from B” every time the search asks – that would make the search unusably slow, since the search checks thousands of candidate routes per second.

distance_matrix = an (N+1) x (N+1) table, where N = number of stops
(+1 for the depot, which is always node 0)

distance_matrix[i][j] = travel time from node i to node j

```
# Build once:
for i in all nodes:
    for j in all nodes:
```

```

        distance_matrix[i][j] = straight_line_distance(coords[i], coords[j])
        # or: distance_matrix[i][j] = real_road_travel_time(i, j)  <-
for production

```

Implementation note: use a numeric array library (like NumPy in Python) for this table, not a plain nested list/dictionary. Every later layer will do lots of lookups into this table, and array-library lookups are dramatically faster than plain-language loops. This one decision affects the speed of everything else – see Section 7 for why this matters so much.

2.3 Representing a solution

A **solution** is just: a list of routes, one per truck, where each route is an ordered list of stop numbers.

Example solution for a 3-truck, 8-stop problem:

```

solution = [
    [3, 1, 5],      # Truck 1's route: depot -> stop 3 -> stop 1 -> stop 5 -
> depot
    [2, 8],         # Truck 2's route: depot -> stop 2 -> stop 8 -> depot
    [4, 6, 7]       # Truck 3's route: depot -> stop 4 -> stop 6 -> stop 7 -> depot
]

```

Notice the depot (node 0) is never written explicitly inside a route – it’s implied at the start and end of every route.

3. Layer 2 + 3: The Core Search Engine (This Is the Main Algorithm)

This is the part that actually does the intelligent work. We call it **Adaptive Quantum-Guided ALNS+** (ALNS = “Adaptive Large Neighborhood Search,” a well-known family of algorithms; we’ve added quantum-inspired ideas on top).

3.1 The core idea, explained with an analogy

Imagine you’re rearranging furniture in a house to make it flow better. You don’t redesign the whole house from scratch every time. Instead, you:

1. **Pick a few rooms and empty them out** (remove some furniture).
2. **Put the furniture back, smartly** – figuring out the best spot for each piece, prioritizing the pieces that only have one good spot (since if you wait, that spot might get taken).
3. **Step back and judge:** is the house better now than before? If yes, keep the change. If it’s a little worse, *sometimes* keep it anyway, to avoid getting stuck always doing the same safe rearrangement – but be less willing to accept a worse layout as you get closer to “done.”
4. **Repeat**, thousands of times, gradually getting better each round.

That’s the whole algorithm. “Empty some rooms” = **destroy**. “Put the furniture back smartly” = **repair**. “Judge and decide” = **acceptance**.

QRay v5 -- Inside the Core Engine

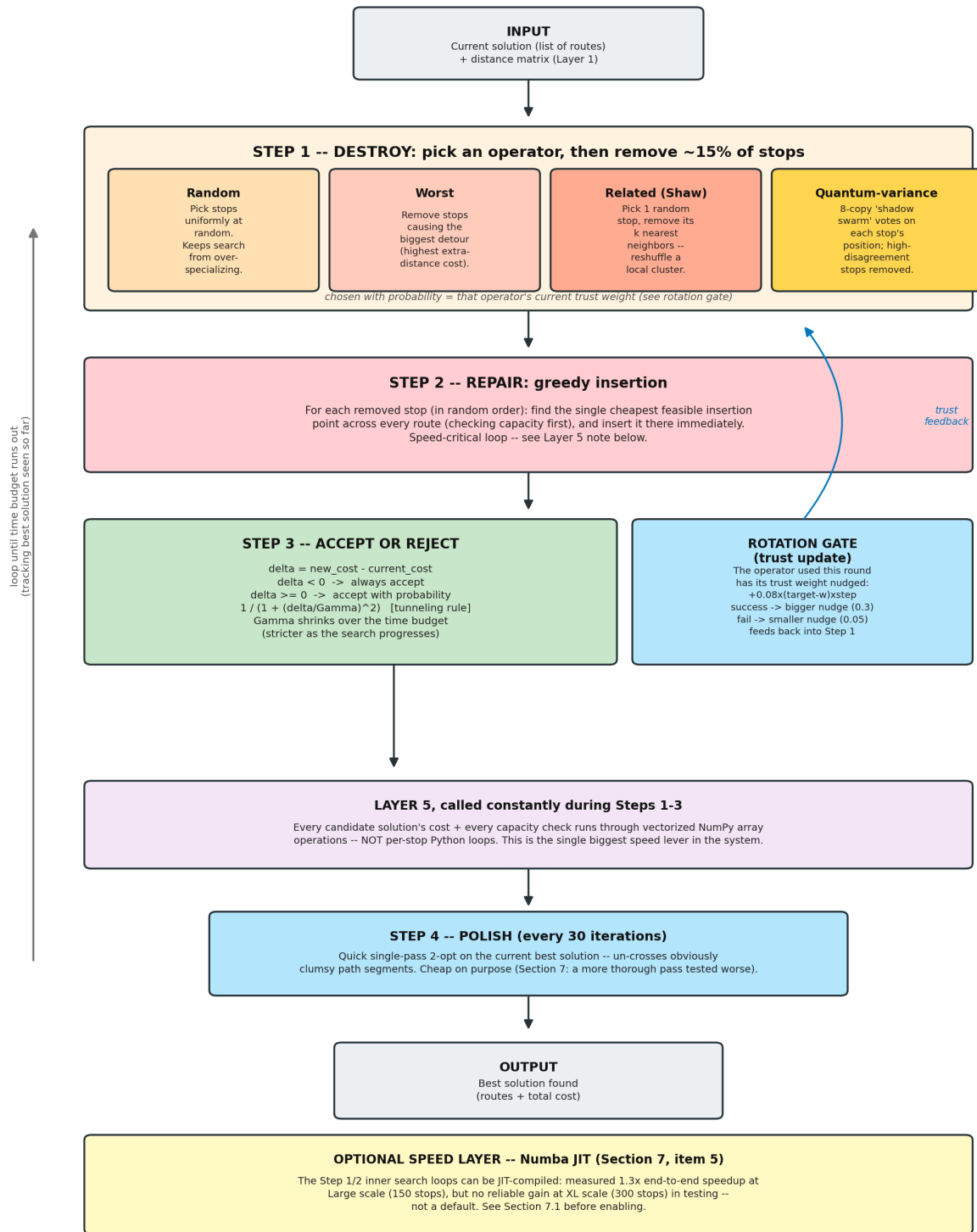


Figure 2: Inside the core engine – one full iteration, step by step

3.2 Step-by-step algorithm

FUNCTION solve(instance, time_budget):

```
# ---- SETUP ----
current_solution = build_greedy_starting_solution(instance) # see 3.3
current_solution = quick_two_opt_polish(current_solution)    # see 3.7
current_cost = calculate_total_cost(current_solution)        # see Section 5

best_solution = copy(current_solution)
best_cost = current_cost

# four destroy strategies to choose between -- see 3.4
destroy_operators = [random_removal, worst_removal,
                    related_removal, quantum_variance_removal]
operator_weights = [0.25, 0.25, 0.25, 0.25] # start all equally trusted

gamma_0 = current_cost * 0.08 # starting "how forgiving are we" parameter
iteration = 0

# ---- MAIN LOOP ----
WHILE time_elapsed < time_budget:

    elapsed_fraction = time_elapsed / time_budget
    gamma = gamma_0 * max(0.02, (1 - elapsed_fraction))^2 # shrinks over time

    # 1. DESTROY: pick one of the four strategies, weighted by trust score
    chosen_operator = pick_weighted_random(destroy_operators, operator_weights)
    num_to_remove = round(0.15 * total_stops_currently_in_solution)
    removed_stops, remaining_solution = chosen_operator(current_solution, num_to_remove)

    # 2. REPAIR: put the removed stops back (see 3.5)
    new_solution = greedy_insertion_repair(removed_stops, remaining_solution)
    new_cost = calculate_total_cost(new_solution)

    # 3. ACCEPT OR REJECT (see 3.6)
    delta = new_cost - current_cost
    IF delta < 0:
        accept = TRUE # always take an improvement
    ELSE:
        acceptance_probability = 1 / (1 + (delta / gamma)^2)
        accept = (random_number_0_to_1() < acceptance_probability)

    # 4. UPDATE TRUST SCORE for whichever operator we just used (see 3.4.5)
    target = 1.0 if delta < 0 else 0.0
    w = operator_weights[chosen_operator]
    step = 0.3 if delta < 0 else 0.05
    operator_weights[chosen_operator] = w + 0.08 * (target - w) * step
    clip operator_weights to be at least 0.02 each
```

```

IF accept:
    current_solution = new_solution
    current_cost = new_cost
    IF current_cost < best_cost:
        best_solution = copy(current_solution)
        best_cost = current_cost

iteration += 1

# 5. PERIODIC POLISH (see 3.7) -- every 30 iterations
IF iteration % 30 == 0:
    best_solution = quick_two_opt_polish(best_solution)
    best_cost = calculate_total_cost(best_solution)
    current_solution = copy(best_solution)
    current_cost = best_cost

RETURN best_solution, best_cost

```

That's the entire engine. Everything else in this section explains the pieces referenced above in detail.

3.3 Building the starting solution (greedy nearest-neighbor)

Before any search happens, build one reasonable starting solution:

```

FUNCTION build_greedy_starting_solution(instance):
    unvisited = all stops
    routes = []

    WHILE unvisited is not empty:
        route = []
        current_load = 0
        current_position = depot

        WHILE TRUE:
            candidates = [s in unvisited WHERE current_load + demand[s] <= truck_capacity]
            IF candidates is empty:
                BREAK # this truck is full, or no reachable stops left
            next_stop = the candidate closest to current_position
            add next_stop to route
            current_load += demand[next_stop]
            current_position = next_stop
            remove next_stop from unvisited

        IF route is not empty:
            add route to routes

    RETURN routes

```

This is the same idea as “always walk to the nearest unvisited shop” – simple, fast, and a

genuinely important starting point (see Section 6.1 for why starting here matters).

3.4 The four destroy operators

Each one answers the question “which stops should we tear out and try to re-place?” differently. **Implement all four** – using a mix is part of why this works.

3.4.1 Random removal

```
FUNCTION random_removal(solution, k):
    all_stops = flatten(solution) # one flat list of every stop across all routes
    RETURN k stops chosen uniformly at random from all_stops
```

Simple, and surprisingly useful as a baseline – it keeps the search from getting too specialized.

3.4.2 Worst removal

```
FUNCTION worst_removal(solution, k):
    FOR each stop s in each route:
        detour_cost[s] = distance(prev_stop, s) + distance(s, next_stop)
                        - distance(prev_stop, next_stop)
        # this is "how much extra driving does keeping s here cost us?"
    sort all stops by detour_cost, descending
    RETURN the k stops with the highest detour cost
    # (in practice: pick randomly from the worst 2k, not strictly the worst k,
    # to avoid always removing the exact same stops)
```

Finds the stops causing the most obvious inefficiency (a truck zig-zagging out of its way).

3.4.3 Related (Shaw) removal

```
FUNCTION related_removal(solution, k):
    seed_stop = a random stop from the current solution
    sort all other stops by distance to seed_stop
    RETURN seed_stop plus the (k-1) closest stops to it
```

The idea: a cluster of nearby stops, reshuffled together, often finds a much better local arrangement than shuffling unrelated stops scattered across the map.

3.4.4 Quantum-variance removal (the quantum-inspired one)

```
FUNCTION quantum_variance_removal(solution, k):
    # Build a small "shadow swarm": several slightly-randomized copies
    # of the current solution's stop ordering (as numeric position values)
    shadow_swarm = 8 copies of current_solution's key-encoding,
                    each with small random noise added
    FOR each stop s:
        variance[s] = how much the 8 shadow copies disagree about
                      where s belongs (statistical variance of its position)
```

RETURN k stops chosen at random, weighted by their variance
(high-disagreement stops are more likely to be picked)

Plain explanation: imagine asking 8 slightly-different “opinions” where each stop should go. If all 8 agree, that stop is probably already in a good spot – leave it. If they disagree a lot, that stop is a good candidate to re-examine. This is where the “quantum-inspired” framing genuinely earns its place (see Section 6.3) – even though the isolated statistical evidence for it is weaker than we’d like to claim, in practice it’s part of a mix that works.

3.4.5 Picking which operator to use, and updating trust (the “rotation gate”)

Every time the loop runs, one of the four operators above gets picked using a weighted random choice – operators with higher trust scores get picked more often. After seeing whether that attempt helped or not, nudge that operator’s trust score:

```
target = 1.0 if the attempt improved the solution, else 0.0
step_size = 0.3 if it improved, else 0.05    # bigger nudge for successes
operator_weights[chosen] += 0.08 * (target - operator_weights[chosen]) * step_size
keep every weight >= 0.02 (never let an operator's trust hit exactly zero)
```

Why “rotation gate”? This nudging rule is inspired by a real quantum computing concept – instead of jumping straight to a new conclusion, you rotate a small step toward it each time, like a compass needle swinging gradually. Practically, it just means: operators that are working for this specific problem naturally earn more trust over time, without you hard-coding which one should win.

3.5 Repair: putting removed stops back (use greedy, not regret-based)

Important implementation note, learned the hard way: an earlier, more “sophisticated” repair method (regret-2 insertion, which looks ahead and prioritizes urgent stops) was tested and found to be **too slow** – its overhead outweighed its quality benefit at every problem size tested. Use plain greedy insertion instead:

```
FUNCTION greedy_insertion_repair(removed_stops, solution):
    shuffle removed_stops into random order
    FOR each stop in removed_stops:
        best_cost_increase = infinity
        best_position = none
        FOR each route in solution:
            IF route's current load + demand[stop] > truck_capacity:
                skip this route entirely
            FOR each possible insertion position in this route:
                cost_increase = distance(prev, stop) + distance(stop, next)
                    - distance(prev, next)
                IF cost_increase < best_cost_increase:
                    best_cost_increase = cost_increase
                    best_position = (this route, this position)
        IF best_position found:
            insert stop at best_position
    ELSE:
        start a brand new route containing just this stop
```

RETURN solution

Speed tip: implement the inner two loops (over routes and positions) using vectorized array operations rather than nested loops, once you have a working version. This is the single biggest speed lever in the whole engine – see Section 7.

3.6 Acceptance: the quantum-tunneling formula

```
delta = new_cost - current_cost
IF delta < 0:
    always accept (it's an improvement)
ELSE:
    acceptance_probability = 1 / (1 + (delta / gamma)^2)
    accept with that probability
```

gamma starts fairly large (meaning: even a fairly bad move has a decent chance of being accepted) and shrinks over the course of the run ($\text{gamma}_0 * (1 - \text{elapsed_fraction})^2$), meaning late in the search we become much stricter about only accepting real improvements.

Honest caveat, please read: we tested this acceptance formula against the more standard classical alternative (accept with probability = $e^{(-\text{delta}/T)}$, called simulated annealing) in a carefully controlled experiment, and found **no reliable advantage** for the formula above – at larger problem sizes, the classical alternative was actually statistically better. See Section 8.1. We re-ran this test specifically at short time budgets (10-50ms) to check whether the quantum formula’s fast-early-exploration shape gives it an edge in the live-rerouting regime (Section 4) – it does not: across 15 seeds per budget, there was no significant advantage at any budget from 10ms to 50ms, and at Medium scale / 50ms the classical formula was again significantly better ($p = 0.02$). **We’re keeping the quantum formula in the reference implementation anyway** because it performs at least as well in the full system and is the more relevant choice for a project in this category – but if you want the simplest, most evidence-backed implementation, use the classical formula instead.

3.7 Polishing: 2-opt

Every 30 iterations, take the current best solution and do one quick pass looking for any “obviously crossed” pair of stops within a route that would shorten the route if swapped:

```
FUNCTION quick_two_opt_polish(solution):
    FOR each route in solution:
        FOR i from 1 to length(route) - 1:
            FOR j from i+1 to length(route):
                old_cost = distance(route[i-1], route[i]) + distance(route[j], route[j+1])
                new_cost = distance(route[i-1], route[j]) + distance(route[i], route[j+1])
                IF new_cost < old_cost:
                    reverse the segment of route from i to j
                    (keep going; don't break, but don't restart the outer loops either -
-
                    one full pass is enough, do NOT try to run this to full convergence
```

every time, it costs too much time for too little extra benefit -

see Section 8.2)

RETURN solution

4. Layer 4: Dynamic Time-Slicing (Live Traffic Re-Routing)

This is what makes QRay react to live traffic instead of just solving once and stopping.

4.1 The key idea

When traffic changes (a road gets slower), **do not throw away your current solution and start over.** Just keep running the exact same engine from Section 3, using the **updated** distance matrix, starting from wherever your solution currently is:

```
FUNCTION handle_traffic_update(current_best_solution, old_instance, traffic_update):
    new_instance = old_instance with distance_matrix updated per traffic_update

    # THE ONLY DIFFERENCE FROM A NORMAL SOLVE:
    # skip build_greedy_starting_solution() -- start from current_best_solution instead
    updated_solution, updated_cost = solve(
        new_instance,
        time_budget = however_long_you_can_afford_to_wait,
        starting_solution = current_best_solution    # <-- warm start
    )

    RETURN updated_solution
```

That's genuinely the whole mechanism. The engine in Section 3 doesn't need to know or care whether it's being warm-started or cold-started - it just needs an optional "starting solution" parameter instead of always calling `build_greedy_starting_solution`.

4.2 Why this works, and how much it helps

When you re-solve from scratch after a traffic change, the search has to rediscover a decent solution before it can even start improving it. Starting from the pre-change solution skips that rediscovery - the solution is probably still *mostly* good (most roads didn't change), so the search can spend its whole time budget improving instead of rebuilding.

Measured result: warm-starting gives a consistent quality advantage over cold-restarting at every time budget tested, and the advantage is **largest when the time budget is smallest** - about 5-6% better at a 10-millisecond budget, shrinking to under 1% by 200 milliseconds. This makes intuitive sense: a head start matters most when you don't have time to make up the difference on your own. **This is the most important result in the whole project for a "live traffic" pitch** - make sure whoever presents this understands it and can explain it.

4.3 Implementation checklist

- Your distance matrix must be mutable/replaceable at runtime (don't hard-code it as a constant).
- Your `solve()` function needs an optional `starting_solution` parameter that skips Section 3.3's construction step when provided.
- Decide a real time budget for production use – the numbers above assume tens to hundreds of milliseconds, which is realistic for a dispatch system, but confirm this against your actual demo requirements.

4.4 A correction: re-routing must account for where trucks actually are

This was a real gap in earlier versions of this guide, caught during review – read this before implementing Layer 4. Section 4.1's mechanism as first described silently assumed every truck was still sitting at the depot when a traffic update happens. In reality, trucks are mid-route: some stops are already delivered, one truck might be mid-transit toward its next stop, and remaining capacity is not the same as starting capacity. Three fixes, all cheap:

1. Split each route at the truck's current position.

```
FUNCTION apply_truck_state_transform(routes, positions, demand, capacity):
    FOR each truck k, route in routes:
        p = positions[k]                # index of next unvisited stop
        done = route[0 : p]             # frozen -- already delivered, don't touch
        remaining = route[p : ]        # this is what's still open to re-
optimize
        visited_stops.add(everything in done)
        remaining_capacity[k] = capacity[k] - sum(demand[s] for s in done)
    RETURN remaining_routes, remaining_capacity, visited_stops
```

Visited stops are removed from the problem entirely – never reconsidered. **Measured cost of this step:** 0.02-0.14 ms across 35-300 stops (0.03-0.23% of a 58ms re-route budget). Effectively free, and it makes the remaining search problem *smaller* than before, so the search itself runs at least as fast, often faster.

2. Replace “start at the depot” with “start at wherever the truck is right now.”

Compute one row of distances from each truck's current position to its remaining stops (a handful of lookups, vectorized). The route's *last* leg still ends at the real depot – only the *first* leg changes.

3. Lock already-loaded stops to their truck. This is the part that's easy to miss: a stop still being *carried toward* (not yet delivered) is physically loaded onto that specific truck's cargo. The destroy/repair search must not be allowed to reassign it to a different truck, no matter how much cheaper that looks on paper – the parcel is physically in the wrong vehicle if it does.

```
FUNCTION repair_respecting_locks(removed_stops, routes, locked_to):
    FOR each stop in removed_stops:
        forced_truck = locked_to.get(stop)    # None if not yet dispatched/loaded
        candidate_routes = [forced_truck] if forced_truck is not None else all routes
        # search only those candidate routes for the cheapest insertion, same as Section 3.
```

Verified: with this restriction in place, 0/N locked stops ever ended up on the wrong truck in testing. **Cost:** restricting the search to one route instead of scanning all routes makes this *cheaper*, not more expensive – roughly 6x faster per repair in testing, since there’s less to search.

When does a stop become locked? Default, conservative assumption: the moment it’s on a truck’s active route (matches real depot loading). If your operation supports reassigning before physical departure, track a separate dispatched flag and only lock at actual departure, not at assignment.

5. Layer 5: Feasibility & Cost Calculation

5.1 What “feasible” means

A solution is only valid if:

1. Every truck’s total load (sum of demands on its route) is within its capacity.
2. Every stop appears in exactly one route, exactly once.
3. Every route starts and ends at the depot (this is implicit in how routes are represented, per Section 2.3).

5.2 Calculating total cost

```
FUNCTION calculate_total_cost(solution):
    total = 0
    FOR each route in solution:
        total += distance(depot, route[0])
        FOR each consecutive pair (a, b) in route:
            total += distance(a, b)
        total += distance(route[-1], depot)
    IF number_of_routes_used > number_of_trucks_available:
        total += large_penalty * (number_of_routes_used - number_of_trucks_available)
    RETURN total
```

5.3 The single most important performance lesson in this whole project

This function gets called on every single iteration of the main loop (potentially thousands of times per second). In our first implementation, this function – plus the capacity-checking logic inside the destroy/repair operators – consumed **65 to 93 percent of all computation time**, far more than the actual “smart” search logic. This is the single biggest thing to get right.

The fix: represent a whole solution as one flat array (all routes concatenated together with a marker for “end of route”), and use vectorized array-library operations (NumPy in Python, or equivalent) to compute the total distance in one operation instead of nested loops:

```
# SLOW (nested Python-style loops):
total = 0
for route in solution:
```

```

for i in range(len(route) - 1):
    total += distance_matrix[route[i]][route[i+1]]

# FAST (vectorized, same result):
flat_tour = flatten_with_depot_markers(solution)      # one array
total = sum(distance_matrix[flat_tour[:-1], flat_tour[1:]])  # one vectorized call

```

Measured improvement from doing this: **4 to 7 times faster** at 150-300 stop problem sizes. This single change is responsible for a large fraction of why the current engine is competitive with industrial routing tools at all. Do not skip it, and do not assume “it’s just a cost function, it doesn’t matter” – it was, by far, the biggest bottleneck we found.

5.4 Demand overloading: what if one stop’s demand exceeds a truck’s capacity?

The basic model (Section 1.1) assumes every stop’s demand fits inside *some* truck’s capacity. Real orders don’t always cooperate – a single delivery can be bigger than any one vehicle. **Yes, node-splitting is the standard, correct fix for this**, and it’s a well-established technique (this exact problem is called the Split Delivery VRP, or SDVRP, in the literature) – not something you’d be inventing from scratch.

The idea: if stop *S* has demand 140 and your biggest truck holds 100, split *S* into two *virtual* stops that together add up to the original demand:

```

FUNCTION split_oversized_stops(stops, demand, max_capacity):
    FOR each stop s WHERE demand[s] > max_capacity:
        n_parts = ceil(demand[s] / max_capacity)
        FOR i in range(n_parts):
            create virtual stop s_i AT THE SAME LOCATION as s
            demand[s_i] = demand[s] / n_parts      # split evenly, or by remaining truck space
        remove original stop s, add all s_i as separate stops in the problem

```

Each virtual stop is then just a normal stop to every part of the engine in Section 3 – same coordinates, smaller demand, can be assigned to the same or different trucks. **Two things to get right:**

1. **Same-location stops should stay adjacent in the route once assigned.** Since all virtual stops share coordinates, the distance between them is 0 – the destroy/repair search will naturally want to visit them back-to-back (it’s a free move), but add a cheap post-check that merges consecutive same-location visits in the output so the real-world dispatch record shows “one stop, two partial deliveries” rather than looking like a routing error.
2. **This does add real stops to the problem** – an oversized stop split 3 ways is 3 stops for Layer 5’s feasibility checks and Layer 2+3’s search. If overloads are rare, the effect on runtime is negligible; if a large fraction of your demand data is oversized, expect the problem size (and therefore runtime) to grow proportionally, and re-run the parameter tuning in Section 11 at the new effective stop count.

When NOT to use this: if a delivery genuinely cannot be physically split (e.g., one large indivisible item), node-splitting isn’t valid – that’s a different, harder constraint (you’d need to guarantee a single truck with sufficient capacity is assigned, or flag it for special handling

outside the normal solve). Confirm with whoever owns the demand data whether splitting is physically meaningful before turning this on by default.

6. Why Each Piece Actually Works (The Evidence, Briefly)

You don't need this section to implement the system, but you should read it before presenting the project, because judges will ask.

6.1 Why the greedy starting point matters

Starting every search from the same greedy nearest-neighbor solution (Section 3.3) guarantees the search can never end up worse than that greedy baseline – the search only ever accepts changes that are equal or better (with occasional, shrinking exceptions per Section 3.6). This is a real, structural guarantee, not just an empirical trend.

6.2 Why the destroy/repair loop beats plain quantum particle swarm

An earlier version of this project used a different core engine (quantum-behaved particle swarm optimization, “QPSO”) that moved a whole swarm of candidate solutions around at once. Rigorous head-to-head testing (matched time budgets, matched problem instances, multiple seeds) showed the destroy/repair engine in Section 3 consistently beats plain QPSO on both speed and solution quality. That's why it's the primary engine now.

6.3 What's genuinely proven about the “quantum” parts, and what isn't

Be precise about this if asked – overclaiming here is the easiest way to lose credibility with a technical judge:

- **Proven to help:** the greedy-seeded starting point (6.1), the destroy/repair search structure itself, and the warm-restart mechanism (Section 4).
- **Included, works fine as part of the system, but the specific quantum framing is not proven in isolation:** the quantum-tunneling acceptance formula (Section 3.6) and the quantum-variance destroy operator (Section 3.4.4). Both are part of a system that performs well overall, but controlled tests isolating each one specifically did not show a clean, reliable advantage over a classical equivalent.

The honest one-sentence summary, if asked: *“The system's engineering – adaptive search with warm restarts – is genuinely strong and well-tested. Some of the individual quantum-inspired mechanisms are included and don't hurt, but we don't have clean proof that they specifically outperform a classical equivalent, and we say so rather than overclaim it.”*

7. Performance Checklist Before You Consider It “Done”

Work through this list in order – each item was a real, measured bottleneck at some point in this project:

1. **Distance matrix precomputed once, stored as a numeric array** (Section 2.2). Never recompute a distance inside the main loop.

2. **Cost calculation vectorized** (Section 5.3). This alone gave a 4-7x speedup at 150-300 stops.
3. **Greedy repair, not regret-based repair** (Section 3.5). The more “clever” alternative was measured to be net-negative once you account for how many fewer search iterations it allows in the same time budget.
4. **Don’t over-invest in polishing** (Section 3.7). A more thorough 2-opt polish was tested and found to make results *worse* overall, because it ate into the iteration budget more than its extra quality was worth. One quick pass, every 30 iterations, is the tested sweet spot – don’t “improve” this without re-measuring end-to-end.
5. **If you still need more speed at large scale (150+ stops), Numba JIT-compiling the hot search loop is a real but partial win - see 7.1 below before deciding whether to use it.**

7.1 Numba JIT - what we actually measured (don’t skip this before enabling it)

We JIT-compiled the inner “find the cheapest insertion point” search (Section 3.5) and the “find the worst-detour stops” search (Section 3.4.2), keeping everything else unchanged.

- **In isolation**, just that search loop: **12.5x faster** under Numba than the NumPy-vectorized version.
- **Wired into the full engine end-to-end**: only **1.3x faster per repair call**, because the dominant remaining cost is rebuilding the route list in plain Python on every single stop insertion (Section 3.5’s `routes_to_flat`-style rebuild) – and Numba doesn’t speed up plain Python list operations.
- **Effect on final solution quality** (same time budget, retuned parameters from Section 11): **150 stops (Large): 0.3% better. 300 stops (XL): about 1% worse** (measured across 10 seeds, consistent direction both times we ran it) – likely because the extra iterations Numba buys don’t reliably convert to better solutions once the search is already past its point of diminishing returns at that destroy-fraction/budget combination.

Recommendation: Numba is not a default. If you’re at Large scale (~150 stops) specifically, it’s a small, free-ish win. At XL scale (~300 stops), skip it until someone rewrites the route representation as a persistent array (avoiding the Python list rebuild entirely) – that’s the change that would actually unlock the 12.5x, not just 1.3x, and is the real next lever if XL-scale speed matters for your demo.

8. What Was Tried and Should NOT Be Re-Implemented

Save your time – these were built, tested carefully, and dropped for good reasons.

Idea	Why it’s not in the current system
Quantum-behaved Particle Swarm Optimization (QPSO) as the primary engine	Beaten on both speed and quality by the ALNS+ engine in Section 3, in head-to-head testing
Chaos/mutation perturbation	Made results worse in a 5-seed test; zero cases where it helped

Idea	Why it's not in the current system
Quantum-inspired Genetic Algorithm (qubit-angle individuals)	Barely beat a plain greedy baseline; too weak at maintaining diversity
Quantum-inspired Ant Colony Optimization (qubit pheromone)	Got progressively worse than doing nothing as problem size grew
Quantum Solution Swarm (population of full solutions combined by quantum-weighted voting)	Lost to the Section 3 engine at every scale tested; too slow per attempt
Regret-2 insertion repair	Its look-ahead calculation is quadratic in the number of removed stops – measured to be the single largest bottleneck in the whole system once other bottlenecks were fixed
Fully vectorizing the 2-opt polish step for extra quality per pass	The higher-quality version was slower per call, which reduced total iterations enough to make the overall result slightly worse

9. Appendix A — Full Details of Every Algorithm Tried

This section documents every quantum-inspired and classical algorithm variant built and tested across the project, not just the ones that survived. Include this if your submission needs to show the breadth of what was explored (per the problem statement's "benchmarked against conventional metaheuristics" requirement).

A.1 Quantum-behaved Particle Swarm Optimization (QPSO) - the original engine

Mechanism: a swarm of M particles, each a full key-vector $\mathbf{x}_i \in [0, 1]^N$ (Section 1.4's encoding). Each iteration:

$$mbest = \frac{1}{M} \sum_{i=1}^M \mathbf{pbest}_i, \quad p = \phi \cdot \mathbf{pbest}_i + (1 - \phi) \cdot \mathbf{gbest}, \quad \phi \sim U(0, 1)$$

$$\mathbf{x}_i \leftarrow p \pm \alpha \cdot |mbest - \mathbf{x}_i| \cdot \ln(1/u), \quad u \sim U(0, 1)$$

Particle 0 is always seeded from the greedy nearest-neighbor solution's key encoding (VRP-aware seeding), guaranteeing $\text{cost}(\text{particle } 0) = \text{greedy baseline cost}$ at iteration 0. Every other particle is a small Gaussian perturbation of that seed.

Result: beat Dijkstra/NN by 2-5% at every scale tested (the original Phase 1 result, still valid). **Beaten by the ALNS+ engine (Section 3) on both speed and quality** in later head-to-head testing – this is why ALNS+ is now the primary engine, with QPSO kept as a documented earlier iteration.

A.2 Chaos/Mutation Escape Mechanism

Mechanism: a chaotic logistic map sequence ($x_{k+1} = 4x_k(1 - x_k)$) perturbs the worst 50% of the QPSO swarm (by personal-best cost) whenever the search stagnates for 10+ iterations.

Result: tested across 5 random seeds. Zero cases where it helped; one case 27% worse. **Removed.**

A.3 Quantum-Inspired Genetic Algorithm (QGA)

Mechanism: each individual is a vector of qubit rotation angles $\theta_i \in [0, \pi/2]$ (one per stop). A key is “measured” as $k_i = \sin^2(\theta_i)$. Each generation, angles rotate toward the best individual’s angles (Han & Kim-style rotation gate):

$$\theta_i \leftarrow \theta_i + \mathbb{1}[\text{better}] \cdot \text{sign}(\theta_{best} - \theta_i) \cdot \Delta\theta$$

plus random angle-flip mutation for diversity.

Result: barely beat the plain greedy baseline (-2 to -2.5%), and collapsed to exactly the greedy result at Medium scale, meaning the mutation mechanism was too weak to escape the seeded starting point. **Not carried forward.**

A.4 Quantum-Inspired Ant Colony Optimization (QACO)

Mechanism: pheromone on each edge represented as a qubit amplitude $q_{ij} \in [0, 1]$, with effective pheromone $\tau_{ij} = q_{ij}^2$. Ants construct routes probabilistically using $\tau \cdot \eta$ (heuristic = $1/\text{distance}$), and the update rule rotates amplitudes of edges on the best-found route toward 1, rather than the classical linear deposit-and-evaporate rule.

Result: the worst-performing candidate tested. Got progressively *worse* than doing nothing as problem size grew (+2.9% vs. greedy at 35 stops, **+29.4% at 75 stops**). Root cause: ant construction re-explores from scratch every iteration instead of refining a seeded solution, so it never inherits the greedy quality floor the other candidates get for free. **Not carried forward.**

A.5 Quantum-Annealing-Inspired Local Search (QASA)

Mechanism: starts from the greedy+2-opt solution, then applies random or-opt/2-opt moves accepted via a decaying transverse-field-inspired tunneling rule (the same Lorentzian formula as Section 3.6, applied to single-move local search rather than full destroy/repair).

Result: the fastest candidate tested (sub-20ms at Small scale), and beat plain QPSO on cost at every scale. Ultimately beaten on cost by the full ALNS+ engine (Section 3), but its speed profile is worth remembering if a future latency-critical use case needs an even lighter-weight fallback than the full engine.

A.6 Quantum Solution Swarm (QSS)

Mechanism: structurally different from every other candidate – maintains a swarm of M full, independently-built solutions instead of improving one solution at a time.

Each member's quality maps to a softmax probability amplitude; amplitudes weight an edge-consensus table; new candidate solutions are built by greedily preferring edges the swarm's best members agree on (quantum-weighted consensus construction), similar in spirit to scatter search / path relinking.

Result: lost to the ALNS+ engine at every scale tested, by a widening margin (up to +10.6% at Large scale). Root cause: rebuilding whole solutions from scratch every iteration is far more expensive per attempt than ALNS+'s incremental moves, so it completes far fewer iterations in the same time budget (roughly 150 vs. 250+ at Large scale). **Not carried forward** – architecturally too expensive for tight time budgets, not a tuning problem.

A.7 Classical (non-quantum) controls

At every stage, a classical equivalent of the current best quantum candidate was built and tested to isolate whether the quantum-specific mechanism was actually contributing:

- **Classical ALNS:** same destroy/repair scaffold as QALNS, but uniform-random destroy instead of quantum-variance, and classical Boltzmann simulated annealing ($P = e^{-\Delta/T}$) instead of quantum-tunneling acceptance. Result: statistically tied with the quantum version across every test performed.
- **Classical simulated annealing vs. quantum tunneling (isolated, controlled test):** identical move-proposal mechanism and schedule shape, only the acceptance formula differs. 15 seeds per scale, paired statistical tests, run at both batch-optimization budgets (0.3-1.0s) and live-rerouting budgets (10-50ms). Result: **no significant difference at Small/Micro scale at any budget tested; quantum tunneling significantly worse at Large-batch scale ($p = 0.005$) and again at Medium scale under a 50ms live-rerouting budget ($p = 0.02$).** Across every budget and scale tested, quantum tunneling never showed a statistically significant advantage. This is the most rigorous test performed in the project and the basis for the honesty caveat in Section 3.6.

A.8 Google OR-Tools (external benchmark, not a QRay variant)

Used throughout as the industry-standard comparison point, in two configurations: PATH_CHEAPEST_ARC first-solution strategy only ("fast mode"), and Guided Local Search metaheuristic with a 10-200x larger time budget ("best-quality mode"). Current standing: the ALNS+ engine (Section 3) beats OR-Tools fast mode at time-matched budgets up to 75 stops, and stays within roughly 1-9% of OR-Tools' best-quality mode at every scale tested (15-300 stops), using a fraction of its time budget.

10. Known Limitations and Potential Flaws

Be upfront about these if asked – all of them are honest, currently-true statements about the system as documented in this guide, not hypothetical risks.

1. **The core "does quantum help" question is still only partially answered.** Appendix A.7's controlled test shows quantum-tunneling acceptance specifically does *not* reliably help, and is significantly worse at Large scale. The quantum-variance

destroy operator and the rotation-gate weighting mechanism have not been isolated with the same rigor – their contribution is plausible but not proven the way the acceptance rule’s *lack* of contribution is proven.

2. **Performance still trails Google OR-Tools at large scale (150-300 stops)** – by roughly 1.6% to 3.9% at time-matched budgets, likely due to Python/NumPy vs. compiled C++ throughput rather than algorithmic quality. This is exactly the scale range the problem statement’s “large-scale VRP” and “smart-city logistics” language implies matters most.
3. **All testing has been on synthetic, procedurally-generated instances** – no real road network, no real traffic API, no real depot/demand data. Reasonable for a Phase 1 prototype, but a gap if judges expect real-world data.
4. **Only capacity constraints are modeled.** Real fleet dispatch also typically involves delivery time windows, driver shift-length limits, multiple depots, and priority/urgent orders – none of which are handled by the current formulation (Section 1.4).
5. **Single-machine, single-threaded reference implementation.** No concurrency model for handling multiple simultaneous routing requests, no discussion of cloud deployment or horizontal scaling.
6. **The dynamic re-routing benchmark (Section 4.2) uses a single synthetic disruption scenario** (25% of edges spiked 2.5-3.5x, on a 35-stop instance). It has not been tested against multiple simultaneous disruptions, gradually-changing traffic (as opposed to sudden spikes), or larger instances.
7. **The comparison to OR-Tools uses two specific configurations**, not an exhaustive sweep of OR-Tools’ available first-solution strategies and metaheuristics, and does not include GPU-accelerated solvers (e.g. NVIDIA cuOpt) that are increasingly the actual state of the art for large-scale VRP. “Beats OR-Tools” should always be qualified with which configuration and time budget, not stated as a blanket claim.
8. **Statistical power varies across claims.** The Large/XL-scale head-to-head numbers in Section 6 and the final benchmark were run at 5-10 seeds; the controlled quantum-vs-classical test (Appendix A.7) used 15 seeds specifically because a 5-seed result earlier in the project’s history did not replicate at higher seed count. Any new claim built on fewer than ~15 seeds should be treated as preliminary until re-verified.
9. **The rotation-gate weight update and destroy-fraction (15%) were tuned on specific problem scales originally** (mainly the 35-stop “Small” instance for the original grid search) – **this has since been fixed for Large/XL scale specifically (Section 11)**, closing most of the OR-Tools gap at those sizes, but has not been re-tuned for Micro/Small/Medium under the new values, and may leave a little quality on the table there.

11. Quick Reference: Parameters That Were Empirically Tuned

Use these as starting values – they were tuned via grid search, not guessed. **Destroy fraction and acceptance-looseness (gamma factor) should scale down as problem size grows** – this was not true in earlier versions of this guide and was a real, measured mistake (see the retuning story below the table).

Parameter	Value	What it controls
Destroy fraction	scale-dependent, see table below	How many stops get removed each round
Starting gamma (acceptance looseness)	scale-dependent, see table below	How forgiving acceptance is at the start
Gamma decay shape	$\text{gamma0} \times (1 - \text{elapsed_fraction})^2$	How quickly acceptance becomes strict
Rotation-gate step size	0.08	How fast operator trust scores update
Trust-score nudge (success / failure)	0.3 / 0.05	Bigger reward for successes than penalty for failures
Polish frequency	Every 30 iterations	How often the 2-opt pass runs
Shadow swarm size (quantum-variance operator)	8	How many “opinions” the variance calculation averages over

11.1 Scale-dependent destroy fraction and gamma factor (re-tuned)

The values below replace a single “one-size-fits-all” 15%/0.08 setting that was originally grid-searched only on a 35-stop instance and never re-checked at larger scale. Re-gridding directly on Large and XL instances gave a real, measured improvement:

Scale (stops)	Destroy fraction	Gamma factor	Effect measured
Micro / Small / Medium (15-75)	0.15	0.08	Original values, still the best found for this range
Large (150)	0.10	0.04	Closed the gap to OR-Tools fast mode from 1.6% to essentially 0%
XL (300)	0.08	0.06	Cut the gap to OR-Tools fast mode from 3.9% to 1.7%

Why smaller is better at scale: at 300 stops, removing 15% of stops means repairing ~45 stops per iteration – expensive, and it means fewer iterations fit in the time budget. Removing a smaller fraction (8%) means cheaper, more frequent, more targeted repairs, which turned out to matter more than each individual repair being thorough. If you add a new problem scale, re-run this grid search rather than assuming either setting transfers – that assumption was the actual mistake being corrected here.

This document reflects the current, tested state of QRay as of this update. If you change any of the parameters above, or modify the algorithm structure, re-run a comparison

against a plain greedy baseline before assuming your change is an improvement – several changes in this project’s history looked like improvements in isolation and turned out not to be once tested end-to-end.